STAT 442/842, CM 762 W25 Assignment 3

DUE: Friday March 7 by 11:59pm EST

NOTES

This assignment has 9 questions for everyone, there are no Manim questions for grad students.

Your assignment must be submitted by the due date listed at the top of this document, and it must be submitted electronically in .pdf format via Crowdmark.

This means that your responses for different question parts should begin on separate pages of your .pdf file. Note that your .pdf solution file must have been generated by R Markdown. Additionally:

Organization and comprehensibility is part of a full solution. Consequently, points will be deducted for solutions that are not organized and incomprehensible. Furthermore, if you submit your assignment to Crowdmark, but you do so incorrectly in any way (e.g., you upload your Question 2 solution in the Question 1 box), you will receive a 5% deduction (i.e., 5% of the assignment's point total will be deducted from your point total).

There are 30 possible marks for all students. That's 30 each, not 30 to share between you all.

Part 1: Generative Art

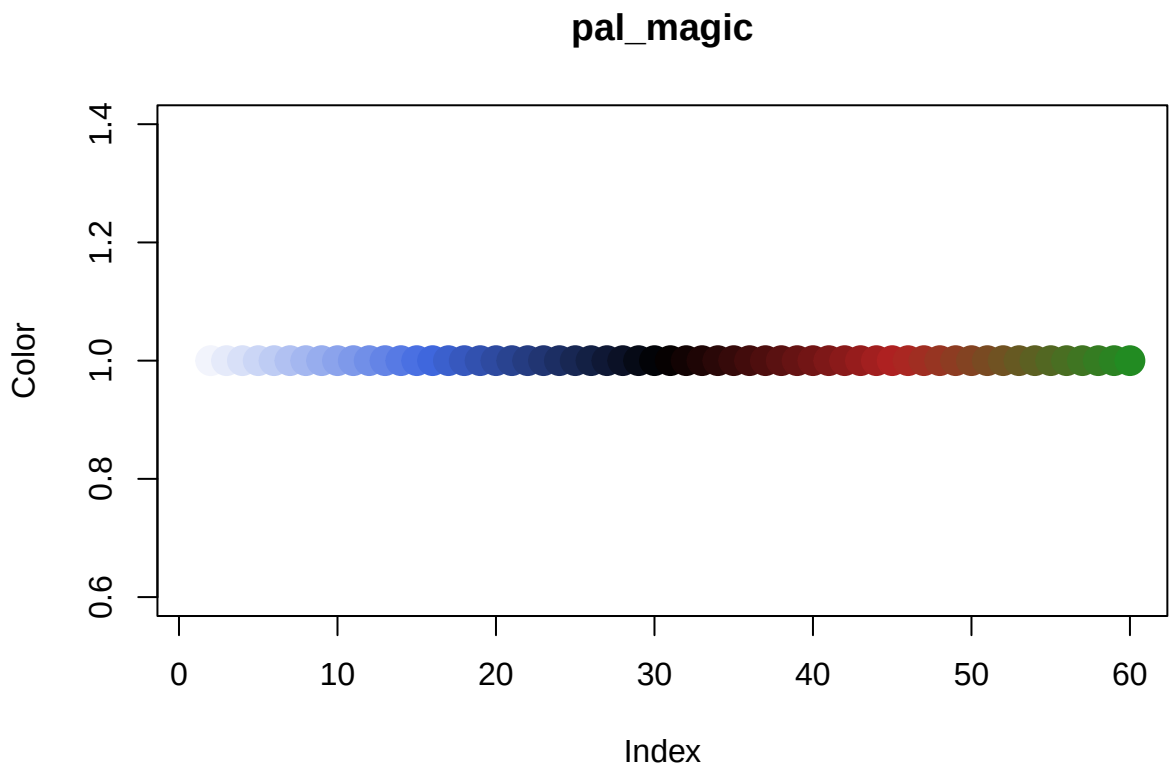
Q1) (2 marks) Using a function from the `RColorBrewer` package, make a palette of 60 hexcodes that interpolates linearly, with default bias and 100% opacity, through white, royalblue, black, firebrick, and forestgreen.

Call this palette `pal_magic` and report the first seven hexcodes.

```
library(RColorBrewer)

#Create the palette using colorRampPalette
pal_magic = colorRampPalette(c("white", "royalblue", "black", "firebrick", "forestgreen"))(60)

#show the first seven hexcodes
pal_magic[1:7]
```

```
## [1] "#FFFFFF" "#F2F4FC" "#E5EAFA" "#D8E0F8" "#CBD6F6" "#BECCF4" "#B1C1F2"
```

```
#show the entire palette in a plot to confirm the colours
plot(1:60, rep(1, 60), col = pal_magic, pch = 19, cex = 2, xlab = "Index", ylab = "Color", main = "pal_
```

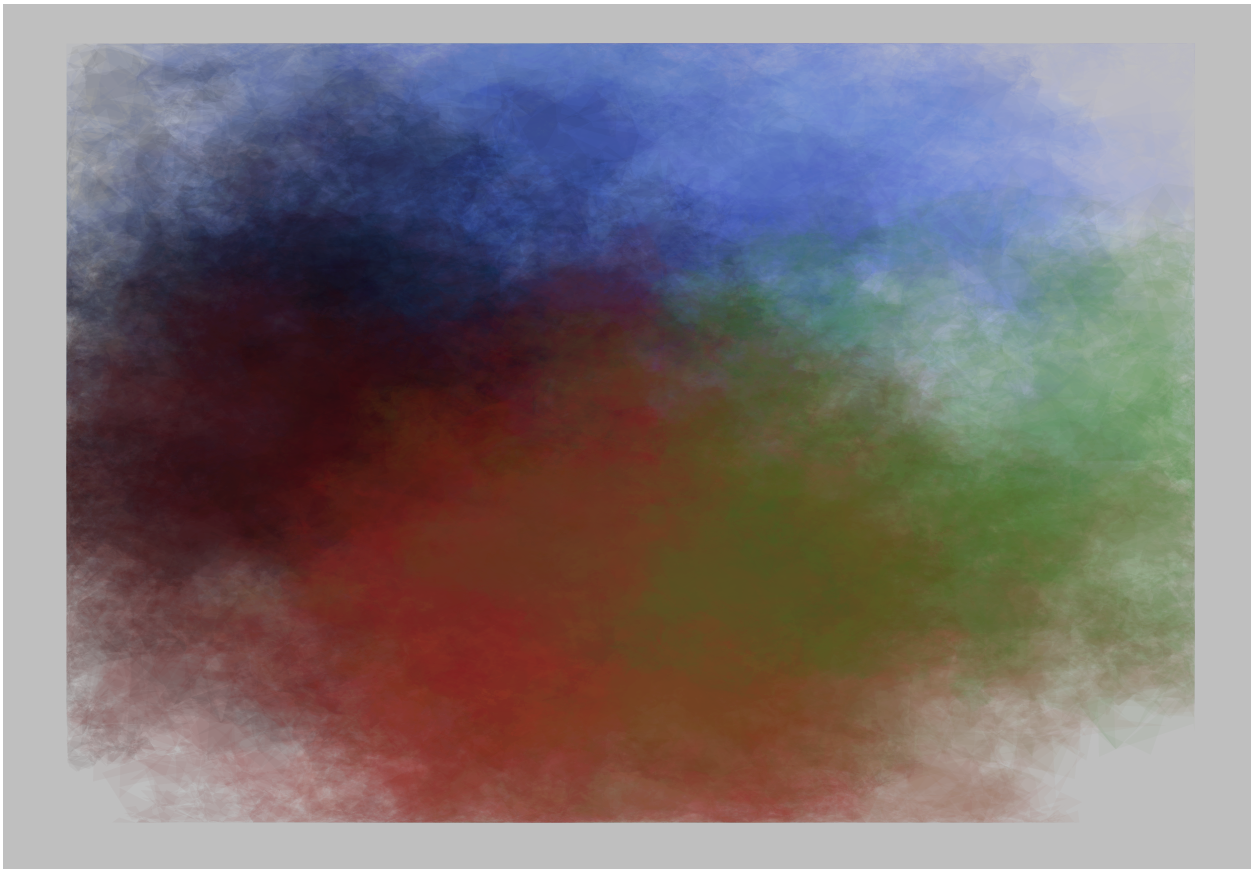


Q2) (2 marks) Using the aRsty package (See https://github.com/koenderks/aRtsy for more guidance), draw a watercolor using `pal_magic`, 1 layer, default depth, a grey75 background, and 50 resolution.

```
library(aRtsy)
```

```
## Warning: package 'aRtsy' was built under R version 4.4.2
```

```
#Draw the watercolor
canvas_watercolors(colors = pal_magic, layers = 1, background = "grey75", resolution = 50)
```



Q3) (4 marks) Using `pal_magic`, the `mandelbrot` package, `geom_raster`, `theme_void`, and `scale_fill_gradientn`, construct an image on a mandelbrot set. The image should be 600 pixels wide, 400 pixels tall, and include a non-trival portion of the set (that is, not all divergent and not all convergent). The x and y limits need to be 0.001 across or less.

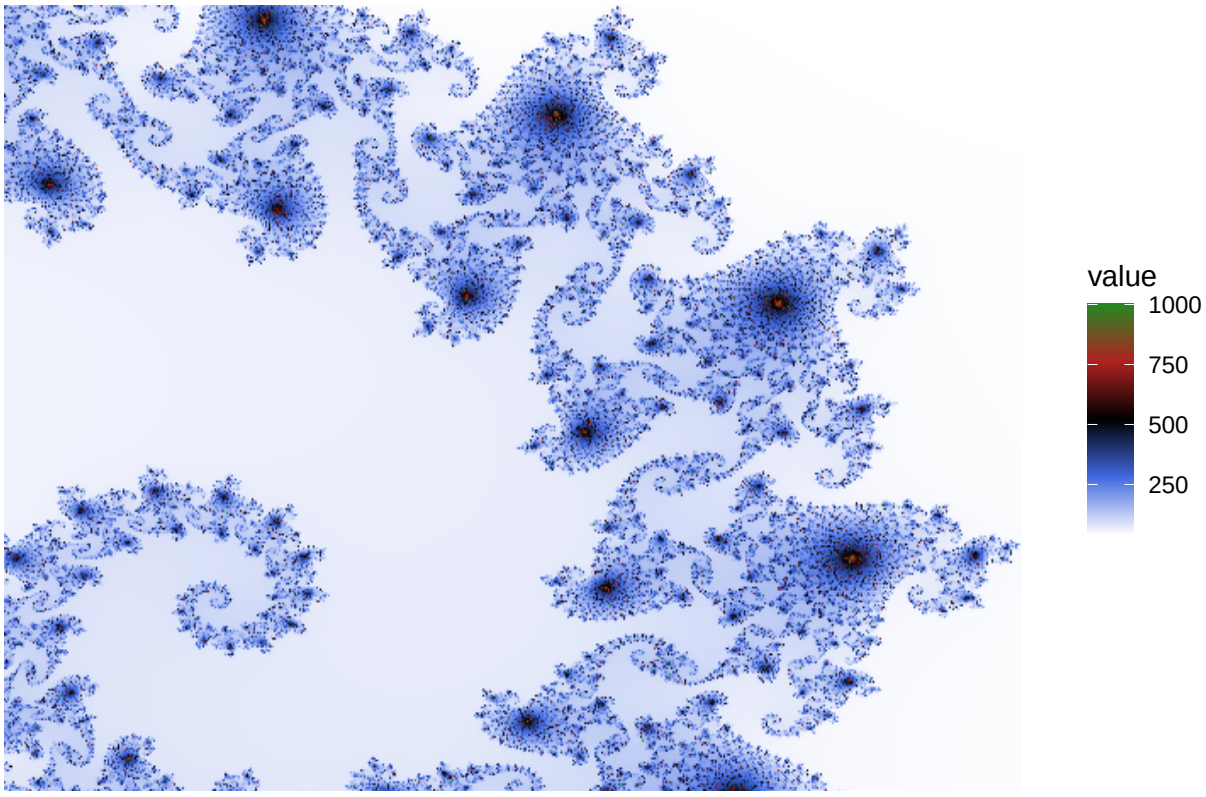
```
library(mandelbrot)
```

```
## Warning: package 'mandelbrot' was built under R version 4.4.2
```

```
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.1
```

```
#Create the Mandelbrot set
mb <- mandelbrot(xlim = c(-0.7435, -0.7425),
                 ylim = c(0.131, 0.132),
                 resolution = list(x = 600, y = 400),
                 iterations = 1000)

mandelbrot_df <- data.frame(mb)

#Create the ggplot
ggplot(mandelbrot_df, aes(x = x, y = y, fill = value)) +
  geom_raster(interpolate=TRUE) +
  theme_void() +
  scale_fill_gradientn(colours = pal_magic)
```



Part 2: Financial Data

Q4) (6 marks) Using the `tidyquant` package, make a chart of the closing stock price for every trading day from Jan 1, 2024 to Jan 1, 2025 of the company "Spinmaster Toys". (Hint: Ticker symbols on the Toronto Stock Exchange end in `.TO`).

The y limits should be 10% above and below the maximum and minimum stock prices within the year, respectively.

If the closing price in Jan 1, 2025 is higher than in Jan 1, 2024 make the area under the line forestgreen at 50% opacity, otherwise make the firebrick at 50% opacity.

```
library(tidyquant)
library(ggplot2)
library(dplyr)

#Ticker symbol for Spinmaster Toys is TOY.TO

# Get the stock data
stock_data <- tq_get("TOY.TO", from = "2024-01-01", to = "2025-01-01")

# Define the color of the area under the line
fill_colour <- if (stock_data$close[nrow(stock_data)] > stock_data$close[1]) {
  "forestgreen"
} else {
  "firebrick"
}

# Create the plot
ggplot(stock_data, aes(x = date, y = close)) +
  geom_line(color = "black") +
  geom_ribbon(aes(ymin=min(close) * 0.9,
                  ymax=close),
                  fill = fill_colour,
                  alpha = 0.5) +
  ylim(min(stock_data$close) * 0.9, max(stock_data$close) * 1.1) +
  labs(title = "Spinmaster Toys Stock Price", x = "Date", y = "Closing Price") +
  theme_minimal()
```


Spinmaster Toys Stock Price
40
35
30
25
Closing Price
Jan 2024
Apr 2024
Jul 2024
Oct 2024
Jan 2025
Date

Part 3: Network Visualization

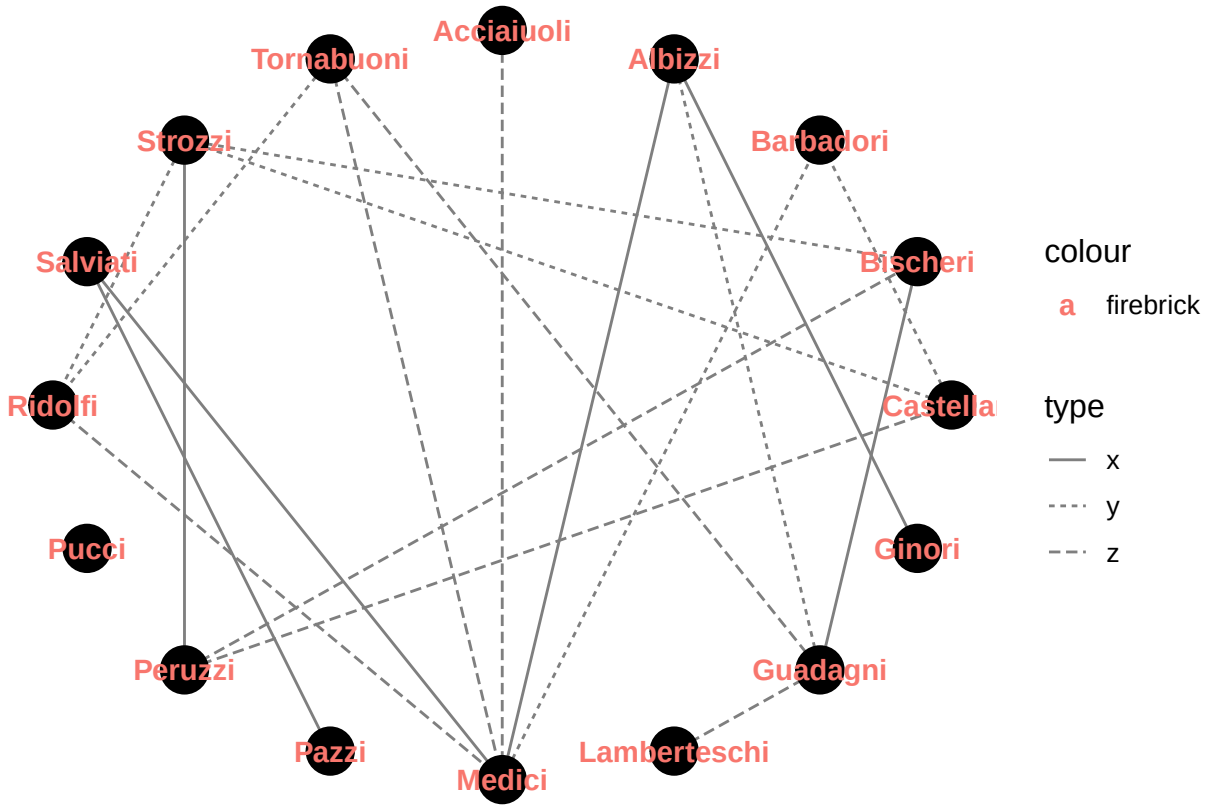
Q5) (5 marks) The dataset `flo` in the `network` package is a network of marriages between elite families in Florence, Italy in the 1200's through the 1400's. After loading the `network` package, you can load this dataset with `data(flo)`.

See https://briatte.github.io/ggnetwork/articles/ggnetwork.html for more guidance.

Draw a network with edges, nodes, and nodetext of `flo` arranged in a `circle` layout. Label the nodes with the names of the families, in `firebrick` color and a bold font face. Use `theme_blank()`.

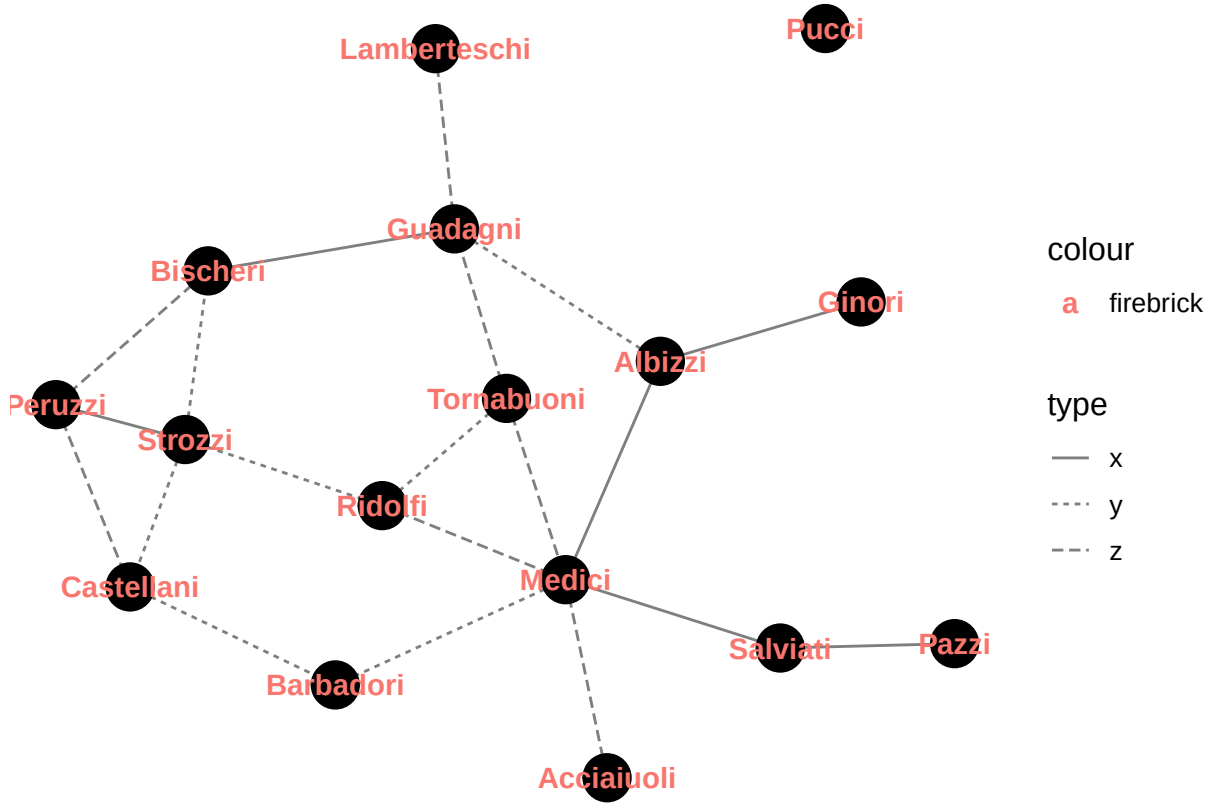
```
library(ggnetwork)
library(network)
library(sna)

# Load the dataset
data(flo)

flo <- network(flo, directed = FALSE)


# Add some attributes to the nodes
flo %v% "family" <- sample(letters[1:3], 10, replace = TRUE)
flo %v% "importance" <- sample(1:3, 10, replace = TRUE)


# Add some attributes to the edges
edge <- network.edgecount(flo)
set.edge.attribute(flo, "type", sample(letters[24:26], edge, replace = TRUE))
set.edge.attribute(flo, "day", sample(1:3, edge, replace = TRUE))

# Create the network
ggnetwork(flo, layout = "circle")
```

```
ggplot(ggnetwork(flo,  layout = "circle"), aes(x=x, y=y, xend = xend, yend = yend)) +
  geom_edges(aes(linetype = type), color = "grey50") +
  geom_nodes(color = "black", size = 8) +
  geom_nodetext(aes(color = "firebrick", label = vertex.names)
                , fontface = "bold") +
  theme_blank()
```


Acciaiuoli
Tornabuoni
Albizzi
Strozzi
Barbadori
Salviati
Bischeri
Ridolfi
Castella
Pucci
Ginori
Peruzzi
Guadagni
Pazzi
Lamberteschi
Medici
colour
a firebrick
type
x
y
z

Q6) (1 mark) Repeat Q5 with the same specifications, but use the `fruchtermanreingold` layout.

```
ggplot(ggnetwork(flo,  layout = "fruchtermanreingold"), aes(x=x, y=y, xend = xend, yend = yend)) +
  geom_edges(aes(linetype = type), color = "grey50") +
  geom_nodes(color = "black", size = 8) +
  geom_nodetext(aes(color = "firebrick", label = vertex.names)
                , fontface = "bold") +
  theme_blank()
```



Q7) (2 marks) Comment briefly (about 30 words) on the relative readability of the network structure between the two layouts of Q5 and Q6, and how this relates to the crossing number.

The Fruchterman-Reingold layout is more readable than the circle layout because it minimizes edge crossings, making connections clearer. It also places highly connected nodes centrally and less connected ones on the perimeter, improving visibility and structure, whereas the circle layout lacks this organization.

Part 4: Sports Visualization

Q8) (4 marks) Use the `hoopR` package and the following code to get the play by play data from the Toronto Raptors home game that happened Monday, Feb 24, 2025.

```
library(hoopR)
```

```
## Warning: package 'hoopR' was built under R version 4.4.2
```

```
dat_pbp = load_nba_pbp(seasons = 2025)
df_pbp = data.frame(dat_pbp)
df_1game = subset(df_pbp, game_id == 401705382)
df_1game_shots = subset(df_1game, shooting_play == TRUE)
df_1game_shots$score_type = as.factor(df_1game_shots$score_value)
```

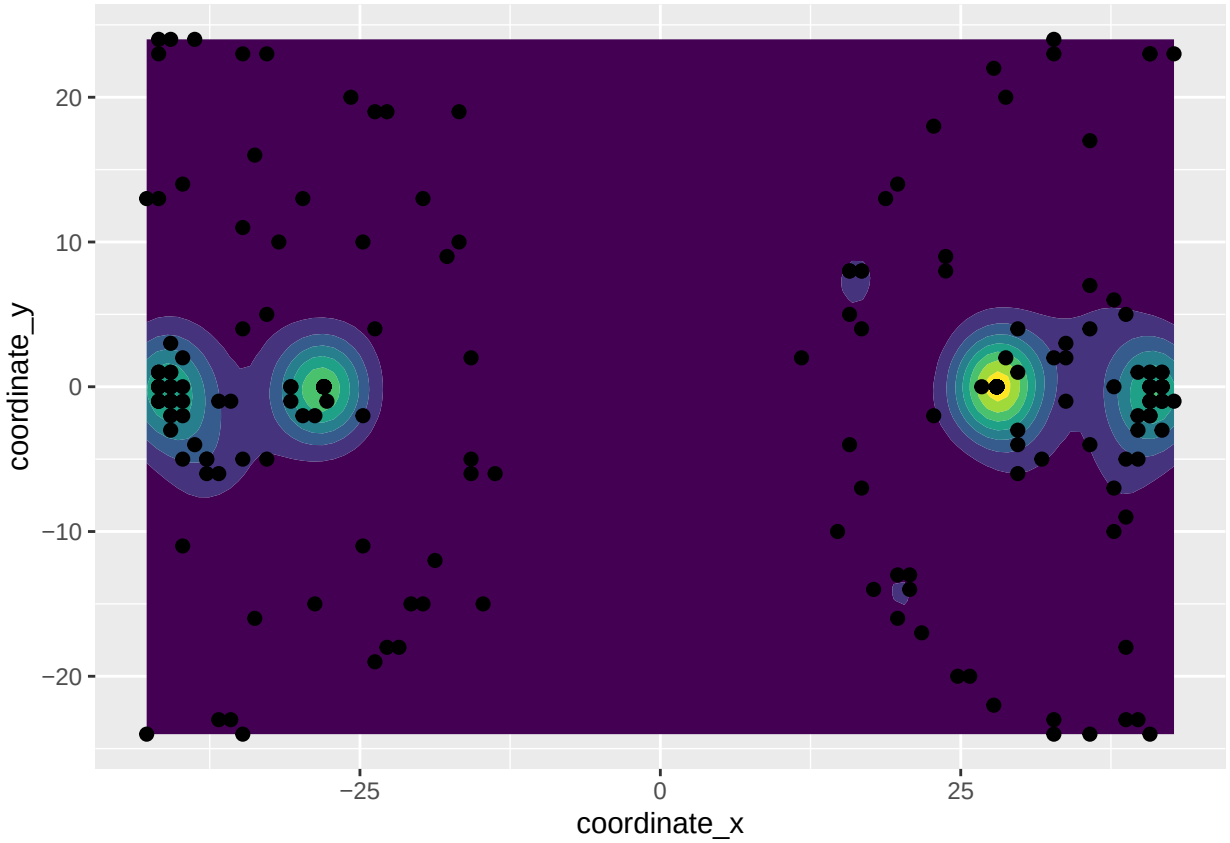
Make a 2D density plot of `coordinate_x` and `coordinate_y` of the shots attempted, using '`geom_density2d_filled`.

- Set the bandwidth `h` to 10 for both x and y.
- Take the legend away.
- Use the default colours.
- Plot the points of the shot attempts on top of the density plot in red with `size=2`.

(See https://r-graph-gallery.com/2d-density-plot-with-ggplot2.html for more guidance)

```
library(ggplot2)

#Create the plot
ggplot(df_1game_shots, aes(x = coordinate_x, y = coordinate_y)) +
  geom_density2d_filled(aes(fill = ..level..), h = 10) +
  geom_point(size = 2) +
  theme(legend.position = "none")
```

```
## Warning: The dot-dot notation (`..level..`) was deprecated in ggplot2 3.4.0.
## i Please use `after_stat(level)` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```


20
10
0
−10
−20
coordinate_y
−25
0
25
coordinate_x

Q9) (4 marks) Using `geom_basketball(league = "nba")` in the `sportyR` package, draw a full basketball court and draw points for shot attempts from `df_1game_shots` on top of the court with `coordinate_x` and `coordinate_y`. Also, map shape to `score_type` and draw the points in Red. Put the legend for the shape aesthetic on the bottom of the plot, and make the legend title be "Points".

Hint: `geom_basketball(league = "nba")` goes where a `ggplot` would usually go. It just draws a basketball court, but it doesn't take in any data, so you'll need to set the data in any geometries that place on top.

```
library(sportyR)
```

```
## Warning: package 'sportyR' was built under R version 4.4.2
```

```
#Create the plot
geom_basketball(league = "nba") +
  geom_point(data = df_1game_shots, aes(x = coordinate_x, y = coordinate_y, shape = score_type), color =
  scale_shape_manual(values = c(1, 2, 3, 4, 5, 6, 7, 8, 9, 10), name = "Points") +
  theme(legend.position = "bottom")
```